

摘要：

OpenAI Codex主管认为，AI时代，“如果你无法解释你在试图实现什么，如果你无法表达你的意图，如果你与你所服务的社区没有连接，如果你没有品味——那做出好的工作会难得多。”他透露，OpenAI已将AI代码审查模型强制应用于所有PR，安全检测全自动、“超越人类水平”。

当AI能写代码、做审查、跑重构，工程师的核心价值究竟剩下什么？

近日，OpenAI Codex主管Tibo Sottiaux在The Pragmatic Engineer播客中表示，在AI快速接管代码执行层的今天，工程师真正难以被替代的，是另一套能力。

“如果你无法解释你在试图实现什么，如果你无法表达你的意图，如果你与你所服务的社区没有连接，如果你没有品味——那做出好的工作会难得多。”

这三件事——**清晰的意图表达、架构品味、系统把控力**——是他给所有工程师的核心建议，也是他在Codex团队内部反复强调的工作原则。

在他看来，AI时代真正稀缺的能力有两类：

第一，深度好奇心与快速理解系统的能力。“在OpenAI做得好的人，都是那种能快速读懂一个系统、钻进一个新代码库、迅速搞清楚它在做什么的人。”他说，这种能力在有了AI辅助之后反而更重要，因为需要吸收和推理的信息量更大了。

第二，清晰的意图表达与架构品味。 Tibo强调，随着AI承担越来越多的执行工作，工程师需要更清楚地定义“盒子”的边界——它应该做什么、满足哪些不变量——而不是纠结于盒子里面的实现细节。“一旦你们对盒子的功能和约束达成共识，盒子里面发生什么，其实不需要太在意。”



AI代码审查已“超越人类”，OpenAI全面强制执行

Tibo在节目中透露，Codex团队早期就开发了专门的代码审查模型，能够深入追踪多层依赖关系，发现人类审查员需要数小时才能定位的逻辑错误。

“这些模型在代码审查上已经达到超人水平——不只是正确性，安全性也是。”他说。目前，OpenAI所有Pull Request都强制经过AI安全扫描，一旦被标记安全问题，PR将被阻止合并，“全部自动化”。

但Tibo认为，代码审查真正的价值从来不只是找bug。“它一直是一种信息交换的仪式，让大家对齐，引发讨论——理想情况下这些讨论应该在写代码之前就发生。”

他的判断是：正确性和安全性的审查将被自动化；而真正值得人类投入注意力的，是围绕意图的讨论——“你到底想做什么？这件事值得做吗？”这个讨论不必发生在Pull Request里，但必须发生。

维护成本下降，架构能力反而更重要

Tibo认为，维护成本的大幅下降，正在重塑工程师的价值分布。

维护本质上是你们为了让系统持续运转而不得不支付的税。但现在，大量维护工作将被自动化——比如第三方依赖升级，模型可以在几小时内扫完整个代码库，直接搞定。以前大家会拖着不做，因为不好玩，但它对安全来说非常重要。

更大的变化在于系统重构。“过去，完整的架构重构可能需要数年。现在这个成本被极度压缩了。”但他同时强调，这反而让架构判断力变得更重要，而不是更不重要：

好的抽象、清晰的边界、明确的不变量——如果你把形状画对了，你就能在‘盒子’内部快速改变任何东西，而不影响其他服务。要为快速迭代而设计。

他提到，OpenAI内部有时会在一个周末内，让100个Agent同时向同一个项目贡献代码。“这种规模的协作以前需要几年时间慢慢扩张，现在可能一个周末就发生了。”

为什么用Rust，为什么开源，为什么支持竞争对手的模型

Codex CLI用Rust编写，在当时是一个反直觉的决定——彼时模型在Rust上的能力远不如Python或TypeScript。

Tibo的解释是：“我们从第一原则出发，把产品界面和Agent本身视为两件不同的事。Agent的核心需要健壮、安全、高效。”他承认用TypeScript也能成功，“但如果你把所有东西写在同一个代码库里，你不可避免地会把东西耦合在一起，然后阻碍后续创新。Rust的边界在这里非常有用。”

开源的决定同样出于务实考量。“你在构建一个编码Agent，当然要把它指向自己。”Tibo说，开源让社区参与成为可能，也让团队招聘变得更简单——“新人入职，他们已经看过代码库了，onboarding基本就完成了。”

至于支持其他模型提供商，Tibo的逻辑更直接：“如果我们不支持，任何人都可以fork一份然后加上10行代码。那样只会让用户去用那个fork，毫无意义。”他说，“我希望靠最好的模型、最高效的模型、最好的产品赢得用户——而不是靠锁定。”



Codex与ChatGPT的合并：从本地Agent到亿级用户

Tibo将Codex并入ChatGPT的技术合并描述为一次重大工程挑战。

“这两套系统完全不同。ChatGPT是全托管云端架构；Codex完全跑在本地。合并的核心问题是：如何把这个本地编程Agent的能力，以足够高效的方式搬到云端，以便纳入每月20美元的Plus套餐，服务数亿用户。”

目前，ChatGPT Work本质上是在云端完整运行Codex的harness，配合一台高性能云端虚拟机。“有些用户已经发现，如果你足够有创意，可以让ChatGPT在里面训练另一个模型，安装Blender做3D建模——它的权限非常开放。”

他透露，整个合并过程中，Codex本身也充当了“记者”角色——它接入了所有Slack频道和文档，自动记录了整个项目的讨论与决策过程，形成了完整的叙事档案，内部称之为“toggle arc”。

在未来方向上，他表示目前的“Work模式切换”只是临时状态：“我们的目标是完全统一——任何在Codex能做的事，在ChatGPT里也能做，反之亦然。”

Tibo自己怎么用Codex

Tibo描述了他的日常工作方式：大量使用手机端ChatGPT Work，在会议间隙用语音输入任务，让Agent生成报告、整理信息、探索代码。

“我现在几乎任何问题都能在30分钟内得到答案。”他说，“无论是某个功能的用户反馈、生产日志的使用量、哪些功能应该下线——Codex都能给我一个初步答案。”

他还提到一个习惯：把白天没时间深入的问题，晚上交给Codex跑，第二天早上看结果。“这让每个早晨都很令人期待。”

对于工程师担心自己的技能被AI取代，Tibo的回应是：“代码是解决问题的工具。如果你真正在乎的是结果和系统运转，那AI让你能解决更多问题，这应该让你成为更好的工程师。”



访谈全文如下：

与 Tibo Sottiaux 共谈 Codex 的构建历程



Tibo Sottiaux 是 Codex 的创建工程师之一。如今，他负责 OpenAI 的核心产品与平台组织，该组织也涵盖 Codex。由于经常慷慨且频繁地发布 Codex 使用额度重置公告，他也是 Codex 最为公众熟知的代表人物之一。

在本期《The Pragmatic Engineer Podcast》中，Tibo 与我讨论了 Codex 是如何构建、又如何持续迭代的。我们探讨了 Codex CLI 为什么使用 Rust 编写并开源，Harness（代理运行框架）与模型如何演进，以及 Codex 为什么支持多家提供商的模型。

Tibo 还分享了 OpenAI 团队如何在整个软件开发生命周期中使用 Codex，包括代码审查、维护和系统重构。我们也讨论了 AI 如何降低修改代码的成本，以及由此带来的一些有趣影响；ChatGPT 与 Codex 的合并；以及 Tibo 如何在自己的工作中使用这些工具。

开场片段

主持人 00:00

你最近在 X 上分享过一个很有趣的故事：你曾参与构建 Google 内部的一款机器人，它有点像 ChatGPT，而且比 ChatGPT 爆火早了一年。当时，Codex 感觉已经不只是一个研究项目了。

你们用 Rust 构建了它，而当时模型对 Rust 的支持并不在主要分布范围内。结果很快就发现：如果愿意投入一些精力，Rust 作为一种语言其实非常适合构建智能体。

Tibo 00:28

当有人加入 Codex 团队时，你会对他们说什么？这里的事情是怎么推进的？

新人最常听到的一句话是：“你问过 Codex 吗？”他们仍会惊讶地发现，基本上任何问题都可以问 Codex。

构建功能很有趣，但维护很痛苦。维护本质上是为了让系统持续运行而必须长期支付的一种“税”。不过，我认为很多维护工作都会被自动化。

代码审查这个概念还会存在吗？代码审查的角色正在改变。

主持人 01:01

Codex 是当今最受欢迎的 AI 编码框架之一，但它是如何起步的？

很多人可能是通过 Tibo 慷慨且频繁的 Codex 使用额度重置公告认识他的。他从 Codex 产品诞生之初就参与其中，并一直领导着更广泛的 Codex 团队。

今天，我们将讨论 Codex 的起源、它为什么使用 Rust 构建并开源；Codex 团队和 OpenAI 内部的编码方式如何变化；维护和重构成本变得极低意味着什么；Codex 并入 ChatGPT 的过程；以及该项目中许多不被充分重视的工程挑战。

如果你想了解 OpenAI 内部的团队如何规划、审查和交付软件，本期节目不容错过。

从童年、数学到创业

主持人 03:10

Tibo，欢迎来到播客。很高兴邀请到你。



Tibo

感谢邀请。再次见到你真好。

主持人

确实很高兴。上次我们是线下见面，这次则通过视频连线。

首先我想问：你是如何进入科技行业的？你是什么时候意识到自己想和计算机打交道的？

Tibo 03:30

这是个好问题。那是很久以前的事了。

我出生在布鲁塞尔。后来，我的父母决定搬离布鲁塞尔，买一栋小房子并进行翻修。不过，那栋房子位于一个偏僻的村庄，周围没有多少事情可做。

我想，当时村里大概只有 200 人。而且其中没有多少人是我想聊天、或者能够成为朋友的对象。

那时我大约 8 岁，于是就开始接触计算机。对我来说，那也是互联网的早期阶段。计算机和互联网成了我学习了解世界的途径。

之后的经历基本都由此而来。我想，我得感谢父母把我带到那个偏僻的地方，让我别无选择，只能对计算机产生兴趣。

主持人 04:20

高中毕业后，你进入大学，开始系统学习这方面的知识，对吗？

Tibo

是的。我在大学学习数学，具体来说是应用数学。我很早就入学了，所以也很早毕业。

有很长一段时间，我觉得自己可能撑不下去，甚至会退学。

在读书期间，我开过一些小公司，也做过小型咨询业务。我为银行工作过，对供应链和应用数学问题非常感兴趣。我尝试将这些能力商业化，并从中学习了很多。

后来，我进入了比利时的创业圈，在那里待了一段时间。之后我搬到伦敦，先后在 Google、DeepMind 工作，如今则来到了 OpenAI，也就是加州。

我很喜欢加州的天气。这个话题我们之后可以再聊。

主持人

你大学毕业后就创办了创业公司，对吗？你很早就有创业冲动。

Tibo 05:13

是的。那家创业公司主要做医药供应链。



我们分析临床试验的供应链，尝试优化和判断：是否应该多生产一些药物？应该把药物运往哪里、从哪里发货？如何避免浪费？通过这些方式提高临床试验的效率。

Tibo 05:35

我们使用的是传统的非机器学习技术，主要是优化求解、蒙特卡洛模拟等方法。

本质上，这是一个随机多阶段优化问题。我们也将这套方法应用在钢铁行业和欧洲电网中。

只要一个问题具备优化问题的结构，我们就会对此感兴趣。直到今天，那家公司仍然存在，我认为他们依旧在做一些非常有意思的工作。不过，随着现代 AI 的发展，这个领域当然也发生了很大变化。

主持人 06:21

很有意思的是，你说“那不是机器学习，只是传统方法”，接着又提到蒙特卡洛模拟、优化和各种算法。

我的感觉是，你会不断深入一个问题领域：面对一个问题，运用自己学过或没学过的数学知识，不断挖得更深。我理解得对吗？

Tibo 06:41

对。这正是我痴迷于应用数学的原因。

理论数学、理论科学和物理学中，有些工作是为了发现未知的事物，或者探索其中的美。它们关乎模式和推动知识边界，但你未必总能知道如何将其应用到现实中。

而现实世界中存在大量有趣的问题。我非常感兴趣的一点是：怎样让世界变得更好？怎样运用复杂的数学方法，优化我们身边的世界？

这就是那家创业公司背后的许多核心理念。

Google、DeepMind 与大语言模型的早期探索

主持人 07:19

创业之后，你去了 Google，2015 年加入 Google 伦敦办公室。

我记得 2015 年进入 Google 是一件竞争极其激烈的事。从行业地位和声望来看，可能就像今天进入 OpenAI 一样困难。

你最初参与的是地图相关工作，后来转到了 DeepMind。能谈谈你当时做了什么，以及为什么会离开一个你显然很喜欢的领域——优化、物流以及这些问题吗？

Tibo 07:46

实际上，我一开始并不在 Google Maps。我最初加入的是一个旨在加快网络和网站速度的项目，尤其是提升移动端网站速度。

当时，Google 正在观察桌面端向移动端的转变，越来越多流量转向手机，因此希望提前布局，资助了多个计划和项目。我参与的就是其中之一。



那段经历非常有趣，因为我们是广告组织内部的一个小团队。这个项目的目标，是在流量向移动端转移的背景下，弥补广告收入可能出现的损失。

我大概做了两年，后来项目被取消了。尽管那是在解决高难度技术挑战方面最开心的一段经历，但我也从中学到很多：例如，没有产品市场匹配度、没有正确的用户、没有有效的反馈闭环时会发生什么；以及，当产品经理说项目进展顺利、但实际上进展很糟糕时，不应盲目信任。

有一天，一位副总裁从加州飞来，告诉我们：“我们要取消这个项目。很遗憾，你们只有几百名用户，这显然不符合 Google 的规模标准。”

令人难以置信的是，当时很多人感到惊讶。

这件事给我留下了一个一直带在身上的教训：始终保持质疑，始终深入思考你正在产生什么影响，以及你所参与项目在整体中的重要性。

Tibo 09:20

之后我去了 Google Maps，那里非常有趣。我参与了评价功能相关的工作。

大约一年之后，我实在无法忽视 DeepMind。它是一个位于伦敦的特别地方，当时有很多令人惊叹的事情正在发生。

那是很早的阶段，AlphaGo 等项目已经开始引起讨论。他们似乎在做非凡的事，真正挑战那些最困难的问题。以我的背景而言，我自然会被吸引。

进入 DeepMind 后，我参与了大量研究基础设施和研究工具的工作。这也是一个我持续了近十年的主题。

我做事的方式一直是：如何构建工具和产品，帮助其他人提高效率，并为他们带来很大的价值。

起初，我是为研究人员构建这类工具。随着时间推移，我开始以越来越通用的方式思考这些问题，最终走到了今天。

主持人 10:19

你最近还在 X 上分享过一个有趣的故事：你参与过 Google 内部一个类似 ChatGPT 的机器人项目，而且这比 ChatGPT 正式发布早了一年。能谈谈这件事吗？这是我以前从未听过的故事。

Tibo 10:44

这是 DeepMind 内部的项目。当时其实有多个相关尝试，Google Brain 也是独立团队，他们也在探索大语言模型。

但大语言模型并不是 DeepMind 的主线。当时 DeepMind 更专注于宏大挑战、博弈问题以及强化学习，而不是语言方向。

不过，有一个团队在推进大语言模型。他们思考的问题是：如果海量文本语料就是一切，会怎样？如果将语言能力推到极限，并持续扩大语言模型规模，是否足以通向通用智能？

这在当时是一个热门争论。

后来有一个团队决定真正全力推进这个方向。对于我们这些为研究构建工具的人来说，很自然会开始思考：我们能用这个模型做什么？怎样把它呈现给研究人员？怎样让他们调试输入与输出？

最终，你会得到一个聊天系统。

我们在内部构建了它，也玩得很开心。最初的模型有点荒诞，表达不太连贯，也谈不上特别有用，但和它们交互、不断尝试依然很有趣。

这个应用像野火一样传播开来，大家都在 DeepMind 内部分享各种对话记录。

Tibo 12:05

它感觉已经不只是一个研究项目，或者说不只是给研究人员使用的项目。

因此，随着时间推移，大家开始希望将它作为外部产品发布。但 DeepMind 并没有为此做好准备。

在 Google 内部，产品发布有一套既定流程，有完整的生产体系和经过多年优化的标准技术栈。它们确实很擅长把产品做好，但在这样的环境中进行真正的创新也很困难。

为什么加入 OpenAI

主持人 12:41

我想回到当时你的处境。假设是在 2023 年或 2024 年：你身处 Google，那里持续发表出色论文、开展优秀研究；你做的事情也非常有趣，正在不断突破此前的边界。

而且你已经在公司内部完成过一次转换。对于一个在当前岗位上感到舒适、满意的人来说——我想你当时应该也是如此——是什么让你仍然会去探索其他可能性，甚至考虑 OpenAI？

Tibo 13:02

我当时确实很舒适。Google 是个好地方。

但我希望能遇到优秀的人，也希望加入一个我真正相信的使命。我希望与那些真正忠于这一使命、深切关心如何以直接而深刻的方式为世界带来积极影响的人一起工作。

我不希望加入一个这样的组织：大家只是做自己的工作，然后把“如何让它变得有用”的问题交给其他人。

我想加入一个能综合考虑所有变量的团队，一个研究与产品真正共同设计的团队。

当时 OpenAI 发展得非常好。ChatGPT 正在起飞。我认识了一些 OpenAI 的人，然后发现：ChatGPT 团队居然只有大约 20 人。

我当时想：“等等，什么？你们只有大约 20 人在做 ChatGPT？这个人数少得惊人。这一定意味着极强的自主性。你们是怎么做到的？如何让只有 20 名工程师的团队维护如此规模的产品？”

随着不断深入了解，我发现那是一个非常出色的团队，有伟大的使命，也有极具天赋和驱动力的人。

这些因素吸引了我。



Tibo 14:23

加入 OpenAI 后，我进入了推理模型发布前的相关工作。

这很符合 OpenAI 的风格：我刚加入不久，就有人说：“我们正在做一件事，准备发布推理模型。这可能是一种新范式，开始冲刺吧。”

大约一个月后，公司就发布了 o1-preview。

这让我感到非常振奋。我想加入的正是这样一个地方：行动迅速、在意影响、与世界保持同步、真正倾听外界。

而这也是我后来在构建 Codex、构建产品时一直坚持的理念：拥有社区，倾听社区，专注于建立极强的反馈闭环。

然后，构建真正值得关心的东西，关注它能为世界提供的实际价值。

Codex 的起点

主持人 15:10

之后，你很快开始参与 Codex。你是在 2024 年加入 OpenAI 的。

能带我们回到当时吗？你加入时，大家对于 AI、大语言模型和编程的看法是什么？

我知道当时有一个 ASE 项目。我们曾在《The Pragmatic Engineer》的深度访谈中谈过它——Autonomous Software Engineer，也就是自主软件工程师，内部称为 A3。

Tibo 15:34

对，内部叫 A3。不过，我们现在已经没有单独的 A3 项目了，它已经成为 Codex。

对我而言，我最初加入时是为研究构建基础设施。我过去做过很多大规模数据存储、数据分析和理解训练过程的工具。

这些年里，我做过很多不同的事，但核心始终是为他人构建工具，让他们更高效，认真对待如何把这件事做好，并通过工具和基础设施创造新的可能。

所以，我加入 OpenAI 后依然延续同样的思路。

随着 o1-preview 以及后续一些模型的出现，我们很清楚：必须利用模型本身帮助我们提升速度。

于是，我开始痴迷于一个问题：模型的限制是什么？我们怎样用这些模型辅助研究本身？

我与研究部门的其他同事一起开始训练模型、构建小型智能体。这些才是真正的 Codex 前身。

我们训练内部模型，让它们非常熟悉 OpenAI 的 Python 代码库，也让它们对架构、代码风格等具备良好的判断力。

当时仅支持 Python。想法是借助它快速构建基础设施，同时帮助研究人员更快地编写代码，从而让我们整体推进得更快。



当不断推进并将事情简化到核心时，我们发现可以非常快速地取得进展、快速学习。

Greg 和 Sam 都非常支持。尤其是 Greg，他非常坚持一点：我们不应只关注自身，也必须关注如何造福世界。

他鼓励我们把这件事不仅视为 OpenAI 内部工具，也要将其变成真正的产品。

这就是我们将这项研究工作与 A3 项目合并、开始构建同一个产品的时候。

之后，我们进行了冲刺，推出了最初的云端 Codex。它当时并没有真正实现产品市场匹配，因为使用摩擦仍然有点高。

我们随后还发布了 Codex CLI，并持续推进。

不过，核心问题始终是：怎样让模型真正帮上忙？

为什么 Codex 选择 Rust

主持人 18:25

你刚才提到，最初你们训练模型理解 Python 代码，帮助构建更好的基础设施。

但后来你们做了一个很有意思的决定：Codex 使用 Rust 构建。当时模型对 Rust 的支持并不像对 Python 或 TypeScript 那样好。为什么做出这个决定？

是因为预期模型会赶上来？还是因为性能更重要？这个选择似乎与大多数其他框架不同。

Tibo 18:48

是的，大多数其他智能体框架并不是用 Rust 构建的，而是使用模型更擅长的 TypeScript、Python 或其他语言。

我们从第一性原理出发，很早就将产品界面与智能体本身视为不同的东西。

因此，我们认为必须以一种稳健、安全、面向效率和规模而设计的方式来构建智能体核心。

过去这些年，我参与过不少项目：一开始只是“一个有趣的东西”，后来却需要扩展到最大型数据中心的规模。在这种情况下，早期技术决策最终会变得非常重要，前提是你不能因此牺牲过多的开发速度。

这是一种取舍。

我们有很多非常高产、出色的 Rust 开发者。我们的内部模型虽然不算特别擅长 Rust，但也并不差。

此外，Rust 在编译期能提供大量验证：静态检查等。这些特性对智能体也很有帮助。

所以很快就变得清晰：如果投入一些精力，Rust 作为语言其实非常适合智能体。

我们的主要关注点是正确性和效率。

主持人 20:07



你的意思是，对于你们而言，值得提前思考产品最终要达到什么状态。

例如，在语言选择上，尽管有了智能体之后重写很多东西比过去容易，但如果一开始就建立正确的基础，仍然能减少返工。

Tibo 20:28

对。我认为即使我们用 TypeScript，甚至 Python 来写，最后也能成功。之后某个阶段，我们可能再重写它。

但对于我们而言，保持智能体本身与产品之间非常清晰的分离，是一个极其重要的原则。

如果把所有内容放在同一个代码库、同一种语言中，最终不可避免会有些松散，彼此的耦合会比应有的程度更深。之后，这会阻碍进一步创新。

因此，从某种意义上说，Rust 的边界非常有用。

为什么 Codex 开源

主持人 21:08

你们做出的另一个有趣决定，在主要 AI 实验室中很独特：将这些组件开源。

CLI、SDK 和 App Server 都是开源的。你们是什么时候、又为什么做出这个决定？

尤其过去常有人开玩笑说 OpenAI 并不“开放”，但这正好相反。其他一些竞争者选择发布闭源框架，而 Codex 是开放的。

为什么要开源？

Tibo 21:36

我们觉得，将代码开源是一件非常酷的事。

因为从根本上说，我们构建的是一个编程智能体。于是我们开始思考：既然有了它，当然会让它指向自身；也许还可以建立一个贡献者社区，让大家用它来改进它，然后我们从中学习。

此外，当时我们很清楚：如果我们成功，开源本身以及代码本身的角色都会改变。

因此，成为这个社区的一部分，而不是与它脱节，是很重要的。

我认为，如果你不亲身经历问题，就很难真正解决问题。

还有一点是，当时虽然我们仍处于很早的阶段，但我们觉得自己对如何做好这件事有一些想法。我们正与训练和研究共同设计这些能力，关键在于以最灵活、最好的方式表达模型能力。

但我们并没有所有答案。

因此，我们愿意公开地展示：一个好的框架是什么样的，我们如何思考它。我们发布了一些非常技术化的深度解析和博客文章，也进行了很多讨论。

我们认为，外面的世界发展很快，有许多非常聪明的人。我们也会从其他开源项目中获得启发。



所以，不如建立一个公平的环境，在这个阶段鼓励大量尝试与探索。

主持人 23:17

如今已经过去一年到一年半了——在 AI 的时间尺度里，这已经非常久。

回顾这段经历，开源给工程和工程团队带来了哪些好处？同时，开源又有哪些困难？我希望听到两方面坦诚的看法。

Tibo 23:59

确实有缺点，开源是有成本的。

好处是：能够公开构建真的很棒。我们的仓库也很小，这很有价值。

每当我们招聘新人，加入 Codex 团队时，他们之前通常已经看过仓库、研究过 Pull Request。入职引导基本已经完成了。

真正的入职方式只是：用 Codex 浏览仓库、询问一些问题。但这不再是什么秘密，因此新人可以立刻开始高效工作。

我们也收到了很多优质贡献。当然，也会收到海啸般的随机内容。

主持人

这显然也是你和所有其他开源项目都会遇到的问题。开源正在发生变化。

Tibo

没错。

对我和团队很多人而言，直接参与社区、直接作出贡献，会带来很大能量。这不只是口头上说“我们重视社区”，而是真正付出努力去做一些事情。我们并非必须这么做。

缺点则在于，开源部分与我们其他代码是分离的。因此，有时我们必须人为划分边界，并在多个仓库之间协作，尤其是在开发一些特别令人兴奋的功能时。

另外，公开构建有时意味着：我们还没来得及发布，别人就已经复制了相关实现。

这会让人有点难受，但也是游戏规则的一部分。既然你选择公开构建，就相当于接受了这样的约定：别人可以复制它。

我们的许可证也非常宽松。不过，当你正在做某个功能，却发现其他人已经将其复制并先行发布，确实还是会有点刺痛。

第三点，和其他开源项目一样，我们会被大量随机贡献淹没，必须承担处理这些内容的额外成本。

不过，这也会促使我们去解决这个问题，我认为这是一件好事。

为什么 Codex 支持其他模型

主持人 25:42



除了开源之外，还有一点让我感到意外：直到最近我才知道，Codex 并不只绑定 OpenAI 的模型，也可以使用其他模型。

从厂商的角度看，这似乎不那么直观。因为我看到的其他厂商在推出 CLI 时，通常都会说“只能搭配我们的模型使用”。

你们为什么决定这么宽松地允许其他模型使用 Codex 的框架？

Tibo 26:16

这对我们来说很自然。

如果你是这个社区的一部分，并且在构建优秀的编程智能体框架，为什么要把它绑定到自己的模型上？做出这种决定会让人觉得有些失望。

这感觉并不对。

我通常会努力做出一些自己能够坦然解释的决定：这就是正确的做法。

开源的逻辑也是一样。

任何人都可以轻松地 Fork 它，然后只改十行代码就加入对其他模型提供商的支持。但如果这样做，你只是在鼓励用户去使用 Fork，接着你会面临额外的维护负担。

而这个 Fork 存在的唯一理由，仅仅是因为你不愿意改十行代码来支持另一个模型提供商。这很没有必要。

为什么不从一开始就支持呢？

另外，我们从给用户选择中获益很多。

今天，你可能很喜欢 OpenAI 模型，并且使用它们非常高效；但明天可能发布一个新模型，你想试试看。为什么要强迫你为了尝试一个新模型，就彻底更换整套工作环境？

我们还可以从这些反馈中获益。也许你喜欢另一个模型的某些特性；也许那个模型效果并不好，但这依然能让我们获得信息。

善待用户和社区，是正确的做法。

我们自己也会尝试其他模型，也会在同一个框架中测试它们。这对所有人都有好处。

这种可选性对于与我们合作的企业也很重要。任何严肃的公司都希望保留选择权，因此我们会坚定支持这一点。

主持人 28:16

我很欣赏这一点，因为它感觉非常坦诚。

它迫使整个公司在每一层都要做到最好：模型层、框架层、开源层，以及可选模型支持。这样你就不能放松下来，说“好了，我们暂时可以歇一歇”。

Tibo 28:30



是的。我希望我们通过最好的模型、最高效的模型和最好的产品来赢得用户。

如果我们把用户锁定在产品里，只因为某一个限制性条件，我认为这也不会吸引最优秀的人来参与构建这个产品。

我们在这里做的是自己最好的工作，非常在意用户体验。无论底层由哪个模型驱动，体验都应该令人愉悦。

我喜欢凭实力取胜，而不是凭登录入口取胜。

Codex 如何在本地与云端运行

主持人 31:50

作为工程师，我一直觉得，只要工具之间有竞争，作为用户就会受益。

我记得微软和 JetBrains 曾有 IDE 之战；云服务商也不断通过功能竞争；如今则有智能体框架和模型之间的竞争。

作为用户，我们拥有更多选择，产品发展更快，我们的声音也能被更好地听到。

说到框架，能介绍一下 Codex 今天是如何运行的吗？

当我启动一个 Codex 任务时，它总是在我的机器上运行吗？它会选择云端吗？会使用沙箱吗？我该如何控制或了解这些？作为工程师，我需要了解多少？

Tibo 32:28

默认情况下，Codex 在沙箱中运行。

如果某条命令需要在沙箱外获取额外权限，Codex 会向用户请求许可。但默认情况下，所有工具执行都发生在沙箱内，并且完全在你的本地机器上运行。

这种模式已经持续一年多了。

不过，这也在不断演进。现在你可以选择在云端运行，它会在受管理的虚拟机中执行。这与 ChatGPT Work 所提供的是同一类虚拟机。

你可以查看这个环境，但它实际上运行在 Kata 容器中，是一个安全环境。所有操作都在该虚拟机内部完成，不会在你的机器上执行。

你的本地设备只负责接收输入和流式返回输出。

这样当然会显著减轻你的 CPU 和机器负担，也能够扩展得更多。

这只是一个开始。未来，使用云端机器会更加无缝。你可能会同时使用笔记本上的部分执行和云端机器上的部分执行。

随着模型越来越强大、越来越有能力，它们可以利用的计算资源和其他资源，远远超过本地机器所能提供的范围。

因此，在某个阶段，仅将执行限制在本地机器上会成为一种约束。



主持人 34:15

本地运行有一个很大的优点，我想这也是我喜欢它的原因。

当然，它也会带来麻烦。如果我同时运行几个智能体，它们会占用 CPU；如果我想合上笔记本，就不能让它半开着继续运行。

我曾去过一家 AI 公司的办公室，看到有人把笔记本半开着。他们问我：“你在运行智能体吗？”我说：“对，我有一个任务正在跑。”

但我之所以喜欢本地运行，是因为我有本地工具、本地 Postgres 数据库，以及各种本地配置。

云端确实很棒，但它没有这些现成环境，或者说设置起来很麻烦。

你们是否在考虑或试验如何处理这些环境配置？

这让我想起我们在 AI 出现之前讨论过的云开发环境。2022 年、2023 年它们很热门，但后来大家开始更多讨论 AI。

我觉得，除大型科技公司外，云端开发机并没有真正普及，因为前期设置成本很高，还要承担维护成本。

Tibo 35:21

是的。对于独立开发者或小团队来说，过去使用云开发环境的收益并不明显。

但有了现在的智能体能力之后，设置的成本几乎可以忽略不计。

如果智能体有能力完成环境设置，它就应该替你完成。

例如，你说自己有本地 SQLite、一个本地服务器、MCP 等环境。要在云开发机中配置完全相同的环境，并让它们保持同步，到底有多难？

如果模型能替你完成这件事，那也许就不难。

我认为，我们将看到完全由云端编排的机器重新兴起，这将让你不再受笔记本电脑限制。

我们已经看到 ChatGPT Work 在这方面取得了巨大成功：它可以在手机上使用。

我每天早晨喝咖啡时，都会通过语音把一堆任务交给它。它会完成这些任务；它可以访问我的日历、电子邮件和 Slack。

能够边走边处理事务，不需要随身带着笔记本，感觉非常棒。

目前，我们交付的能力仍比较极端：执行依然发生在笔记本上。但如果你无需一直让笔记本保持开启状态，那将非常美好。

Harness 与模型如何共同进化

主持人 36:41

你们过去是如何改进 Codex 的？



我记得自己第一次使用早期版本的 Codex 时，可以和它交谈，它也能完成一些任务。

但比如我说：“做这个改动。”它完成了改动，可我有单元测试，它却没有运行。

之后几个月，不知道具体是哪天，它开始自动运行测试。

这些变化是怎样实现的？你们是在改进运行脚本、指令，还是某种启动脚本？还是在改进模型？

作为开发者，我该如何理解每个版本的进步——框架与模型之间分别发生了什么？两者又是什么关系？

Tibo 37:26

这是个好问题。

从某种意义上说，框架总是比模型领先一点。

主持人

这是什么意思？

Tibo

意思是：模型具备某些能力，但你会给它加上一些“拐杖”，让它能够以可靠、高效且符合用户预期的方式完成任务。

这就是框架的作用。

它提供护栏、安全性，提升效率，并让智能体更容易被引导、控制和调整。

框架通常还负责所谓的“开发者消息”，也就是在每轮开始时注入上下文中的指令。这显然会影响智能体在整个回合中的行为。

你看到的很多效果，都是框架与模型共同作用的结果。

最初，你可能会发现它不运行测试，因此必须提醒它运行测试。之后，我们训练出更好的模型，它们更能理解：当用户提出请求时，用户真正希望它完成什么。

于是，随着模型进步，你不再需要告诉它运行测试。

长期来看，我们会看到系统提示词越来越短，框架本身也越来越简化。

主持人 39:08

在 Codex 团队内部，你们是否会设定具体目标？

比如说，现在 Codex 作为“框架加模型”的组合，在某件事上做得不够好，或者会犯一些愚蠢错误。

作为工程团队，你们如何推进下一版本的 Codex？

作为开发者，我会觉得模型有点像魔法，它自然会变得更好。我知道你们肯定有反馈渠道。



但你们同时也在构建框架和工具，这大概是团队负责的部分。

你们甚至如何设定目标？传统软件开发中，你会说“我们要做这个功能”，然后去实现，因为你知道怎么做。但这里的开发过程对我来说有些模糊。

Tibo 39:44

确实如此，这也是我们为什么共同设计大部分事情。

这是研究团队与工程团队之间协作的过程，尤其是与构建核心智能体框架的工程团队协作。

我们总是在讨论：今天我们在什么方面做得很好？什么方面做得还不够好？还有哪些事情值得做，因为它会成为一个很酷的产品功能？

之后，我们会判断：这个问题应该通过框架改动解决，还是通过模型改动解决？

如果要通过模型改动解决，那么多久能做到？一个月？三个月？六个月？

我们会据此推进。

根据模型能够多快解决问题，以及需要在哪个训练层面调整，我们有时会决定完全不在框架里做改动，而是等待模型解决它。

我们会全面使用智能体。我们用智能体分析大量反馈，归纳主题，帮助我们进行讨论和确定优先级。

我们会分析编码领域的使用情况，也会分析金融、通信、营销等其他领域——如今用户已经在这些领域使用我们的智能体。

我们大致知道自己在不同子类别中的表现，也始终在推动能力边界。

有一点很有意思：随着预训练模型变得更好，随着整体模型变得更好，整个系统的能力都会提升。

当然，也有些事情是我们特别投入更多注意力的。

OpenAI 内部如何使用 Codex 开发软件

主持人 41:12

你提到“全程使用智能体”。我们能否谈谈 Codex 团队的软件开发生命周期？

在过去、AI 出现之前，新工程师加入团队时，通常会被告知流程是这样的：

产品经理或团队有一个想法；我们制定计划；一起估算；拆分工作；编写代码；测试；代码审查；发布；使用功能开关；然后进入值班支持。

这就是过去的方式。

现在，当有人加入 Codex 团队时——他们显然已经在开源部分有所贡献——你会如何告诉他们，这里的事情是怎么完成的？

Tibo 42:11



如果他们是完全的新手，我会先把他们介绍给优秀的人。

然后，他们最常听到的一句话是：“你问过 Codex 吗？”

在 OpenAI，Codex 默认接入了几乎所有东西。它可以访问 Slack、文档和所有代码。

新人仍会感到惊讶，因为你基本上可以问它任何问题，它经常都会给出非常好的回答。

理解一个项目的状态、谁在负责某件事、某项决策为什么做出，最简单的方式往往是直接问 Codex。因为 Codex 在内部几乎知道所有这些事。

这也是我们为什么会在公开频道中完成大量工作，会以相对广泛的权限共享文档：这样每个人都能访问信息，智能体也能够检索、推理和整合这些内容。

我们还有一些对团队生产力和协作非常有帮助的工具，目前尚未发布，但很快会推出，其中一部分会在 DevDay 上亮相。

这些能力能让你与团队其他成员保持高度同步，帮助你快速理解现状，也帮助你迅速产出成果。

一般建议是：关心用户，关心产品的一致性，关心模型及其发展方向。

如果你正在构建一个一万行代码的“拐杖”，只是为了绕过你知道模型未来会修复的缺陷，那么你可能做错了。

我们有一套原则，但此时它更像是一种团队文化和精神。新人加入后，会通过整个团队继承这些根基。

主持人 44:02

假设我有一个想法，我认为它很好。我先和 Codex 讨论，可能也会和一些同事讨论。

我决定把一个很酷的新功能作为自己对 Codex 的第一个、或者第一个重大贡献。

接下来怎么做？

显然，我会用 Codex 编写代码，也会测试并确保它正常运行。从那里开始，流程是什么？

你们是否仍有代码审查的概念？是否有 AI 代码审查、验证、分阶段发布、上线后验证？

毕竟 Codex 面向数百万用户。它刚刚跨过 2,000 万活跃用户的里程碑；而如果进入 ChatGPT，面向的用户规模还会大得多。

Tibo 44:44

无论是发布到 Codex 还是 ChatGPT，流程都出人意料地相似。

尽管 ChatGPT 面向的是约十亿活跃用户，而且还在增长，但你依然可以提交一个 Pull Request、做一项改动，并在第二天甚至当天发布，直接覆盖十亿用户，而且一切都没问题。

我们真正做的是培养强烈的主人翁意识与责任感。

人们被充分授权去做改动，甚至是大的改动。



通常会被要求提供的是：证明它会受到良好欢迎的证据；证明它是值得加入的内容；证明它值得长期维护的证据。

而维护的成本也已经大幅下降。因此，我们对这些事情的思考方式与两三年前有所不同。

另外，我们会尽可能自动化。

代码审查、部署和回归检测等大量流程，基本都已自动化。

这样，大家就可以把注意力集中在想法本身、它将如何帮助用户、整体体验的一致性、智能体的能力和持续改进上。

我们有一长串希望实现、但尚未完成的事情。

我们的北极星方向是：一个令人愉悦、易于使用的个人 AGI。

它应当了解关于你的、它需要了解的一切；能够访问正确的资源；有时能代表你采取存在风险的行为；之后向你发送推送通知，让你进行确认。

用户应该能够深刻理解它。它要了解你的日程、目标，可以主动为你提供帮助，并且使用起来应该极其自然。

你应当能通过自然语言、语音来控制它。也许它还应理解你的情绪；如果拥有摄像头画面，它应该能以最自然的方式理解你。

它不应该是一个有十个不同按钮和复杂配置的东西。AGI 应该简单易用。

代码审查的角色正在改变

主持人 46:34

你刚才简略提到了审查，但我想回到代码审查的话题。

你曾在 Google 工作，参与过 Google Maps 这类面向数亿用户的产品。Google 以严格的代码审查文化闻名。

我想他们有两层代码审查：一层是语言正确性审查。他们长期以来一直在行业内不断完善这套流程，也相信它的有效性。

你怎么看这部分正在发生的变化，尤其是人工审查？

长期以来，直到一两年前，我都会说代码审查有很多好处：知识共享、第二双眼睛、降低“巴士因子”，因为其他人也会理解代码；以及围绕架构而不仅仅是代码展开讨论。

但现在代码量多得多了。

代码审查的价值是什么？在哪些情况下依然重要？在你们这样领先的团队中，人工或开发者参与审查的价值在哪里？又在哪些地方可以放心交给智能体？

Tibo 47:50

代码审查的角色正在改变。



我早期参与 Codex 的一个项目，就是和研究团队一起开发一个代码审查模型。

我们的目标是让它达到这样的水平：它能够发现逻辑和推理中的错误，而人类可能需要数小时、甚至多人投入才能发现同等程度的问题。

因为这可能需要向下深入三四层依赖关系，理解文档是否有误，理解某个第三方依赖的实现是否与预期不同，进而发现你的不变量没有被维持，最终导致 Bug。

除非你本身是那个库的专家，否则很难注意到这些问题。

我们开发了这些代码审查模型，也发布了它们。如今，它们发现错误、进行深度验证的能力已经成为主线模型的一部分。

在基准测试中，它们在代码审查方面已经超越人类。

这不仅适用于正确性，也适用于安全性。

例如，模型能跨越非常复杂的上下文进行推理，最终指出：“这里存在一个严重的安全漏洞。”

如今，在 OpenAI 的所有 Pull Request 中，这已成为强制要求。如果系统标记出安全问题，我们会阻止 Pull Request 合并。这一切都是自动完成的。

我认为，代码审查过去始终关乎正确性，确保功能正常运行；但它也是一种信息交流的仪式，使人们保持同步，并鼓励讨论。

理想情况下，这些讨论应该在写代码之前进行，但有时它们只会围绕代码发生，因为一旦代码合并，它就进入生产环境并开始实际运行，而之后你还必须维护它。

因此，其中还包含社会协作的因素。

我认为，正确性、网络安全和安全审查都会被自动化。

真正会保留下来的，是围绕意图展开的讨论：你到底想做什么？你尝试做的事情是否正确？

我认为，这种讨论可以发生在 Pull Request 之外，不必围绕代码本身。

也许这会帮助我们更清晰地区分：哪些地方真正需要讨论，而哪些地方过去只是因为没有现在这样的工具，才不得不通过代码审查来进行讨论。

这会发生变化。

过去，代码审查是一个强制机制：因为在代码合并、进入生产环境前，你必须进行这些讨论。

但我认为，现在有其他方式来完成这些讨论、共同设计，并确保意图正确。代码本身的重要性将不再那么高。

主持人 50:26

回想我经历过的所有代码审查，当然有许多美好的记忆：我们有过很好的讨论，或者我学到了很有价值的东西。

但坦率地说，也有很多时候它非常令人痛苦。



我想推进自己的工作，就不断催别人：“能审一下我的代码吗？”对方说：“不行，我现在很忙。”我再说：“但我真的需要这个来解除阻塞。”然后对方就被迫切换上下文。

代码审查一直都有好与坏。

我想，无论未来如何，始终都会有好处和坏处，只是它们在转移。

其中一个好处是，工程师不必再把注意力花在那些并不真正需要人工判断的基础问题上。

Tibo 51:24

是的，这会节省时间。

我们将逐渐看到这样的情况：团队先就一个“盒子”及其整体契约达成一致。

只要对资源使用、数据访问、安全等有严格保证，那么盒子内部发生什么，可以是任何事情。

你不必关心盒子内部的每一项实现。

真正需要达成共识的是：这个盒子到底做什么？必须满足哪些不变量？

我认为，这些才值得进行高质量讨论，也许可以由你最喜欢的智能体辅助完成。

一旦这些内容明确，在盒子内部修改任何事情，都不再需要重新进行同样的讨论。

这能真正保护和节省人的注意力。

维护与重构成本下降

主持人 52:07

维护成本已经降低。

在 Google、Uber、创业公司等各种组织中，维护一直都是热门话题。构建功能很有趣，但维护却令人痛苦。

很多时候，我们后来才意识到：“也许这个东西根本不值得做。”

在 Codex 和 OpenAI 内部，你看到维护成本如何变得更低？这会如何改变大家构建系统的方式、目标、抱负，以及自定义工具等方面的决策？

Tibo 52:30

维护本质上是一种长期支付的税，用来保证系统持续运行。

过去需要维护，未来仍然需要维护。

但我认为，很多维护工作会被自动化。

比如，你有一个第三方依赖，需要升级版本。过去你可能会想：“我得记得处理这件事。”但现在，如果有好的变更日志，代码文档也足够完善，模型可以推理整个改动。

它可以在数小时内扫描整个代码库并完成升级。



过去，人们可能会把这件事搁置，因为它不够有趣；但它对业务和项目其实很重要，尤其涉及安全漏洞时。

你希望保持最新状态，希望及时应用所有补丁。

我认为，这些都会被完全自动化。

因此，维护工作中很大一部分会近乎免费地完成。

另外，过去如果你想彻底重构，或者需要新的架构，以便为新的权衡、新的工作负载认知或新功能腾出空间，突然发现现有系统限制极大，必须全面重构——这是一项代价极高的工程。

有时甚至需要多年。

我认为，这件事现在也大幅加速了。

因此，犯错的成本正在下降。

但与此同时，软件工程中那些经典原则仍然很重要。例如，良好的抽象仍然非常有帮助。

这又回到前面提到的“盒子和不变量”：如果你划出了正确的边界，就能够更快修改盒子内部，而不会影响其他服务或基础设施。

我认为，设计时必须考虑如何支持极快的迭代与变化。

主持人 54:34

我记得曾和 Peter Steinberger 交流，那是在他加入 OpenAI 前。我们谈到了 OpenClaw 以及他如何思考这类系统。

他说自己不怎么阅读代码，但我能看出来，他一直把架构放在脑中。

他告诉我，他会频繁重构，思考如何实现模块化，如何让 100 名贡献者各自构建内容而不互相干扰。

我从你的话里听到的是：这种关注、规划和结构化工作也许变得更加重要了。

过去，这可能主要是架构师、资深工程师或经验丰富的人在做，其他工程师则在周围构建较小的部分。

但现在，听起来所有工程师都需要在构建软件时关注这些问题、提前做好规划。

Tibo 55:16

是的，GPT 模型在这方面也越来越好。

它们越来越擅长思考长期维护、良好架构等问题。

这是一种自然的下一步：不只是关注代码质量，例如某个文件中的代码是否干净；而是思考架构本身是否正确，是否能降低长期维护负担，是否能为未来产品或功能的扩展与变化留出空间。

这种长期工程实践，模型正开始变得非常擅长。



我觉得很有意思的是：我们构建的软件，其生命周期正在以更快的速度推进。

以前，一个项目起步时，也许只是你自己和几位工程师。之后才慢慢增加工程师。若项目非常成功，一年后可能会有 50 名甚至 100 名工程师加入。

你有时间预见这一切，有时间让人加入，有时间准备文档和其他流程。

但现在，突然有 100 个智能体同时参与一个项目，这种事情可能一个周末就会发生。

因此，我们进入这一切的速度，比过去快得多。

工程师如何适应高速变化

主持人 56:42

但你和 OpenAI 的同事是如何面对这一切的？这种变化不会扰乱你们的认知吗？

你的意思是，你已经在这个行业工作很久了，有几十年经验。过去我们习惯了一种节奏，而现在节奏快得多。

一年前你还在做的事情，现在可能已经不做了，因为模型已经擅长完成它。

你如何与这种变化和解？

我相信，有些与你的软件工程能力相关的事情，你过去非常擅长，但现在可以交给智能体。

这不会带来一点失落吗？

我们刚刚说过，自己的开源功能被别人复制会让人有点难受。同样地，如果你以前很擅长重构，而现在智能体可以做；今天也许是架构设计，未来模型可能也非常擅长，你会不会觉得有点可惜？

Tibo 57:43

我认为，这其中确实存在一种手艺层面的感受。

有时我仍会打开编辑器，亲自写一些代码。那种感觉很好。

我对一些夜晚有很美好的回忆：坐在 Vim 前不断编码，喝着可口可乐 Zero，不必思考其他任何事情，只专注于眼前的问题。

但我认为，真正重要的是进入心流、解决问题。

我发现，这里的同事，以及我交流过的所有人，都在非常迅速地适应。

如果你的心态是：代码只是解决问题的工具，而现在你能解决的问题多得多，那么变化就是积极的。

过去，如果你想做个基准测试，却不确定结果如何，你可以直接让它在后台执行。可能只需要 30 秒，就能获得可靠数字，从而做出更好的权衡。

只要你真正关心结果、关心系统是否正常工作，这应该会让你成为更好的工程师。



在 OpenAI，这使我们能更高效地运行推理，更有效地利用计算资源，并将这些能力交付给世界。

因此，大家都专注于这一点：以过去不可能达到的速度解决重要问题。

到目前为止，我还没有遇到有人说：“这不好，这不有趣。”

主持人 59:10

我理解的是，如果你有足够宏大的问题，有远超过今天、明天或下周所能解决的问题，这就不是问题。

因为当你在某个方面变得更高效后，你只会继续向前。

这就像很多创业公司：它们的抱负总是远大于自己当下能完成的事情。

Tibo 59:32

没错。我们绝对不缺问题要解决，而且我认为短期内也不会缺。

我们在数学突破、科学突破、让世界变得更好、真正为人类构建产品，以及以深刻人性的方式解决人们最重要的问题方面，还有很长的路要走。

这正是我们在这里的原因。

Tibo 01:00:00

再回到编程和那些深夜。

我刚才讲的，可能有些浪漫化。事实上，我也经历过很多深夜，试图重构某件事，三个小时后才发现这条路根本走不通，必须从头开始。

那非常令人沮丧。

当然有快乐的时刻，但也有那种代码无法编译的时刻，你不断想：“为什么它还是编译不过？”

主持人 01:00:30

我相信你也经历过这种情况：时间已经很晚了，你该睡觉了，但因为有一项任务只做了一半，根本睡不着。

我记得自己甚至会梦到代码。

而如今，我在为自己的业务开发软件时，通常不会再有那种“只完成一半”的状态。因为我可以让智能体完成任务，然后离开；要么任务已经完成，要么它在运行，要么我已经得到失败的证据。

一切都加速了。

Tibo 01:01:13

我现在仍然会有类似的情况。



很多人都会这么做，我自己也是如此：有时我会思考一些更大的问题，可能来自白天的对话，或者我还没来得及深入研究。

于是，我会让 Codex 在夜间研究这些问题。

第二天醒来查看结果时，我会非常兴奋。因此，每个早晨都令人期待。

主持人 01:01:34

我觉得这就形成了一种长时间运行任务的模式。

当然，你可以使用 `/goal` 命令来持续运行任务。这也是几个月前才加入 Codex 的能力。

回到“框架是拐杖”这个概念，`/goal` 是否就是一种拐杖？它是否是为了让模型在很长一段时间里专注于单一目标而设计的？

Tibo 01:01:56

是的。`/goal` 是必要的，它让模型能够长期围绕一个目标保持正轨。

它甚至可以让模型在面对真正困难的问题时，持续运行数天或数周。

但随着新一代模型出现，我们看到的是：你不再需要 `/goal`，也不再需要围绕它的额外框架。

你可以直接告诉模型：“去工作一周。”它确实能做到。

ChatGPT 与 Codex 的合并

主持人 01:02:21

说到难题，以及你们并不缺少难题，有一个从外部看来很有意思的发布：你们所说的“合并”。

Codex 出现在 ChatGPT 中。

从工程师的视角看，我会说这件事中等程度地有趣。我们已经在用 Codex；现在可以在 ChatGPT 应用里打开它，很好。

但我通常不会在应用里使用 ChatGPT，而是直接进入 Codex。

不过，我和 OpenAI 的很多人、包括你团队的人交流过，他们说这项合并前进行了很多准备，面临很多持久且艰巨的挑战。

能否介绍一下：这究竟是一个多大的项目？你们需要做什么？为什么难？Codex 及其他工具又如何帮助你们以过去难以实现的方式完成它？

自从合并推出后，你不断分享 Codex 用户数量增长的数据，增长速度比过去快得多。我猜这里解决了巨大的扩展问题。

Tibo 01:03:25

这次合并有很多挑战。

首先，两者的技术栈完全不同。



ChatGPT 是完全托管的、基于云端的产品。所有东西都运行在我们的系统上，我们以传统的大规模、高效率方式存储和构建它。

而 Codex 是完全本地化的。

因此，合并的核心在于：如何让这个本地编程智能体获得同样的价值与能力，并围绕它构建一个产品，使其能够惠及更广泛的人群。

这也是我们加入 OpenAI 的原因：希望让非常广泛的全球人群从中受益。

因此，这是一个令人兴奋的探索过程：怎样构建一个云端版本，使它在本质上具备非常相似的能力；同时，又能够以高效的方式服务数千万、数亿用户，从而可以将它包含在 Plus 订阅方案中。

ChatGPT Work 本质上是在云端运行完整的 Codex 框架，并配备一台云端计算机。

它实际上是一台非常强大的机器。人们已经发现并展示了它能做什么。

如果你的提示词足够有创造力，可以让 ChatGPT 在其中训练另一个模型；也可以让它安装 Blender，进行 3D 建模。

它的权限非常宽松，能访问互联网，是一台功能强大的机器。Codex 在这台机器上运行。

这就是我们通过 ChatGPT Work 交付的能力。

这其中涉及很多系统层面的挑战。

团队完成得很快。显然，Codex 帮助我们更高效地分析并构建大量基础设施，也帮助我们处理 Codex 与 ChatGPT 之间出现的很多细微差异。

例如，合并插件架构、合并库，并真正朝着统一系统迈进。

我们的目标是：用户不应该觉得有些事只能在 Codex 中做、不能在 ChatGPT 中做，或者反过来。

我们希望构建一个统一产品，让你以自己希望的方式，访问同样的智能。

Tibo 01:06:05

整个过程中还有一件很有趣的事：Codex 也充当了记者。

它记录了整个过程中的步骤、争论和讨论。团队之间针对如何实施、如何命名、何时引入、以什么形式引入、什么与什么合并，进行了非常热烈的讨论。

我们考虑过许多不同方案。

因此，这个项目有很有趣的新闻记录元素：Codex 随着时间推移记录了整个过程。

这也逐渐被称为 OpenAI 的“切换开关故事线”。

我们引入了 Work 开关。围绕它是否是正确做法，也有过大量讨论。但后来，我们逐渐真正喜欢上了它。



随着时间推移，我们会进一步合并这些能力。

我们正朝着完全统一的方向前进。目前的状态只是暂时的：当你处于 Work 模式时，能力会更强；但长期来看，我们会将这些能力带给所有 ChatGPT 用户。

Tibo 如何使用 Codex

主持人 01:07:13

你个人如何使用 Codex？

你的工作配置是什么样的？你如何使用智能体、处理哪些任务、管理哪些事情？

相关地，我问过 Peter Steinberger 我应该问你什么。他说：“你一定要问他，他参与那么多项目，日历像俄罗斯方块一样排满，但通常还显得很愉快。他是怎么做到的？”

Tibo 01:07:33

我的日历还好。

如今我能够完成更多事情，是因为有了 Codex 这样的技术。

我已经把很多工作转移到手机上，通过 ChatGPT Work 完成。

每当我想记录一件事，我就直接把它发出去。我经常使用语音输入。

每当我有问题，以前我可能会写下来，之后再研究，或者委托给别人；现在，我会直接在 ChatGPT Work 中把任务发出去，然后得到一份报告。

它拥有许多自定义技能和自定义指令，现在已经非常适合为我生成我能高效吸收的报告、幻灯片和代码探索结果。

因此，每当我在会议之间有空档，你可能会看到我对手机说话。

正如我前面提到的，我们在公开频道中完成许多工作。Slack 中有很多信息，Notion 和 Google Docs 中也有很多信息。

所以，几乎不存在我觉得无法提出的问题。Codex 至少可以对任何问题进行第一轮思考。

无论是分析某个功能的公众情绪，查看生产日志以了解某项功能的使用量，列出应该淘汰的、没有获得用户认可的内容，还是了解某个团队正在做什么——任何问题，我都可以在大约 30 分钟内获得答案。

所以，我用它来做所有事情。它在各种意义上都是我的个人智能体。

周末时，我也常做一些代码探索或构建原型，尝试想象产品的未来。我会和团队中的其他人一起做这些事，不一定总是同一个团队。

有时，一天之内，我就可以构建出一些原本只存在于脑海中的东西。

比如有一天早上醒来，我会想：“我们应该探索一下构建这个东西意味着什么。”



然后，我就能把这些想法表达出来，在一天内做出一个能展示给其他人的东西。大家可以据此思考、批评，并希望从中获得启发。

这并不意味着我们一定要发布它。

更像是：我把这个想法从脑海中释放出来，然后继续做其他事情。

这是一个非常神奇的时代，也非常赋能。

给软件工程师的建议

主持人 01:10:19

最后，如果一位软件工程师、AI 工程师，或者任何构建软件的人，希望培养所需的技能和经验，以便有机会加入 Codex 团队、OpenAI 或一家 AI 创业公司，成为一名善于使用这些工具的优秀构建者，你有什么建议？

人们经常问：我应该从理论开始吗？基础知识有多重要？还是只要非常擅长使用工具就可以？

Tibo 01:10:41

我认为有两件事很重要。

第一，是对事物如何运作抱有深度好奇心；第二，是训练自己快速理解事物的能力。

事情会持续变化，但在 OpenAI 表现极其出色的人，通常是那些能够迅速理解一个系统、进入新的代码库并理清其脉络的人。

当然，如今智能体能够帮助完成这些工作。

你需要吸收的信息非常多，因此，能够理解并推理这些信息非常重要。

其中很大一部分，其实是提出好问题：事物到底是如何运作的？

不断追问“五个为什么”，持续向下挖掘。这样你会学得非常快。

第二点，是与社区保持同步，或者说，与那些你试图为其解决问题的人保持同步。

并不是每件事都在解决一个直接的问题。有时，你解决的是一个对另一群人有用的问题，而他们最终也在为人类问题的解决作出贡献。

但你必须清楚地理解用户的品味、需求和要求；必须能够清晰思考，并持续训练这种思维清晰度。

这对我来说非常重要。

如果你无法解释自己想实现什么，无法解释自己的意图，无法与某个社区建立联系，也没有相应的品味，那么你就很难做出伟大的工作。

主持人 01:12:05

太棒了，Tibo。非常感谢这次对话，真的很精彩。



感谢邀请。

节目总结

主持人 01:12:16

我一直想和 Tibo 深入聊一次，很高兴终于实现了。

我很欣赏 Tibo 不仅谈论开源的好处，也坦率谈到了它的缺点。

尤其是，当你正在公开开发某项功能时，竞争对手可能复制这个功能，并在你正式发布前抢先上线。这确实会令人不舒服。

另外，开源还会带来大量低质量贡献，而你仍然必须以某种方式处理它们。

另一个有趣的观点是，Tibo 说框架总是比模型领先一步。

从内部来看，Codex 团队认为自己的工作，是通过框架、工具和设置指令为模型构建“拐杖”。随后，下一版本的模型会被训练得不再需要那么多拐杖。

坦率说，作为开发者，这听起来有一点让人泄气：我构建的东西，下一版模型可能天然就会知道，于是就能被移除。

不过，我怀疑事情并不只是构建这些拐杖，也包括构建模型会使用的工具。下一代模型应该不会重新发明 MCP 协议、技能或插件。至少我希望不会。

我也很喜欢了解从内部看，合并 ChatGPT 和 Codex 是怎样的过程。

这意味着要将原本完全本地化的编程智能体 Codex，整合进一个托管式云端技术栈；同时还要做到足够高效，使其能够纳入 OpenAI 每月 20 美元的订阅方案。考虑到 20 美元能购买的计算资源并不算多，这并不容易。

还有一件很有趣的事：Codex 本身充当了整个项目的记者。由于它出现在所有 Slack 对话和文档中，因此能够记录所有重要争论和决策。

我不得不承认，这一部分有一点“老大哥”的感觉：AI 一直在看。

但这很可能会成为未来创业公司的新常态。我还没有决定自己对此究竟是什么感受。

最后，我很认可 Tibo 给工程师的建议：保持好奇心，快速理解系统，并与自己正在服务的人群保持同步。

从 Tibo 的分享中听到基础能力依然如此重要，也令人感到安心。

请查看下方节目说明，其中包含关于 Codex、Claude Code、Cursor 构建方式及其他相关主题的深度解析。

如果你喜欢本期内容，请在你使用的播客播放器中留下评分。这对我 and 节目都意义重大。

谢谢，我们下期再见。



风险提示及免责条款

市场有风险，投资需谨慎。本文不构成个人投资建议，也未考虑到个别用户特殊的投资目标、财务状况或需要。用户应考虑本文中的任何意见、观点或结论是否符合其特定状况。据此投资，责任自负。

 93  收藏

分享到:  

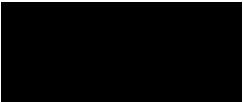
写评论

请发表您的评论

 表情  图片

发表评论

1 条评论



正在加载 .